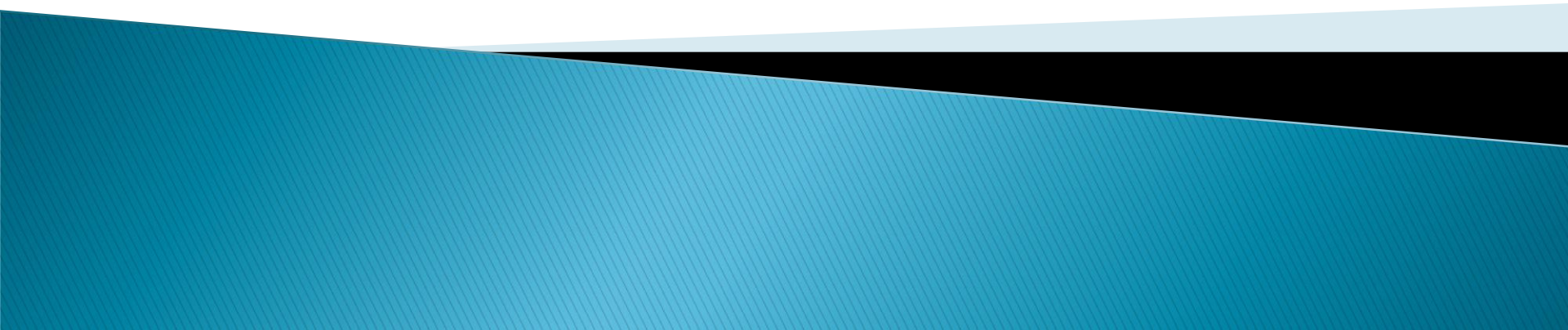


# COMPUTAÇÃO GRÁFICA

AULA 10: Remoção de Faces Ocultas

Prof. Paulo Bressan

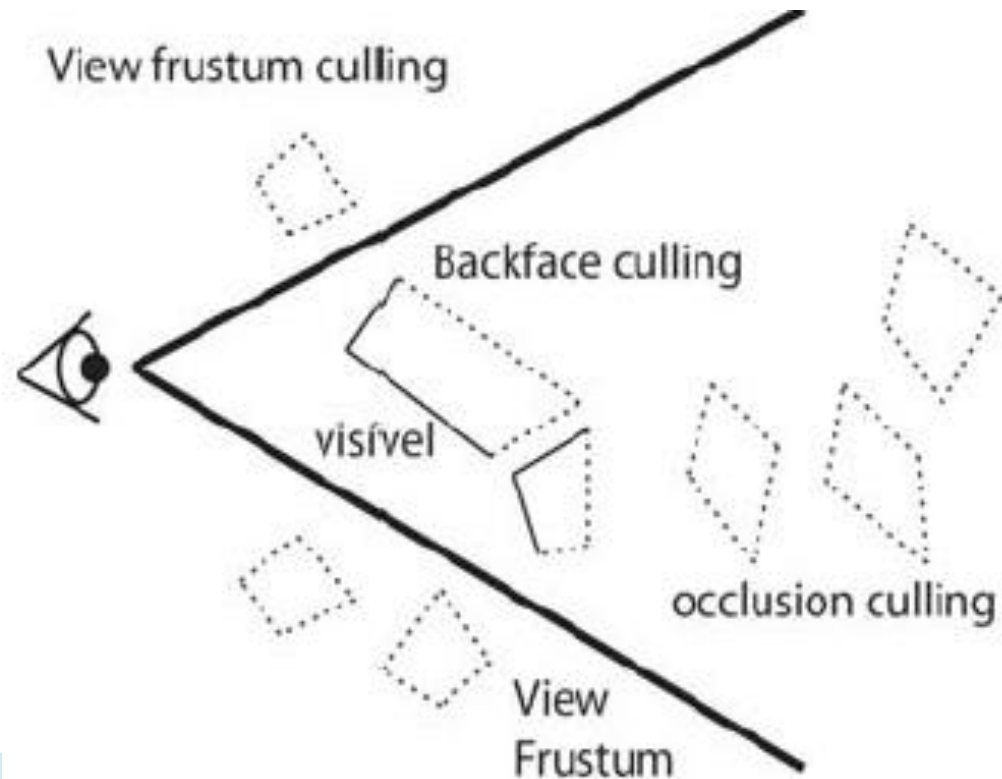


# Remoção de Faces Ocultas

- ▶ Dado um conjunto de polígonos, qual é visível em cada pixel? Quem está na frente?
- ▶ Há muitos algoritmos diferentes para isso, duas grandes classes:
  - Espaço do objeto: computação opera nas primitivas
  - Espaço da imagem: computação opera no nível do pixel
- ▶ Pode-se operar em qualquer espaço do pipeline de visualização, pois todos eles mantêm a informação de profundidade.

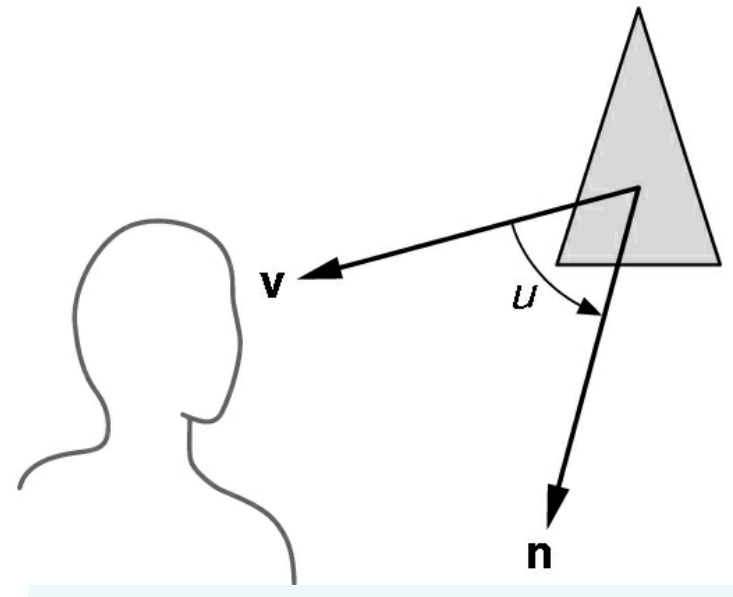
# Remoção de Faces Ocultas

- ▶ Determinar a visibilidade dos polígonos antes de enviá-los para placa gráfica (*culling*):
  - *Back face culling*;
  - *View frustum culling*;
  - *Occlusion Culling*;
  - Célula / Portal.



# *Back face culling*

- ▶ Eficiente
- ▶ Não renderiza o que não pode ser visto
- ▶ Face é visível se o ângulo entre observador e a normal da face satisfaz:
  - $-90 \leq \theta \leq 90$
  - $\cos \theta \geq 0$
  - $n \cdot v \geq 0$



# Desafios

## ▶ Eficiência

- perde-se tempo sobrescrevendo pixels ou rasterizando coisas que não podem ser visualizadas

## ▶ Precisão

- resultado precisa ser correto e se comportar corretamente quando a câmera se move

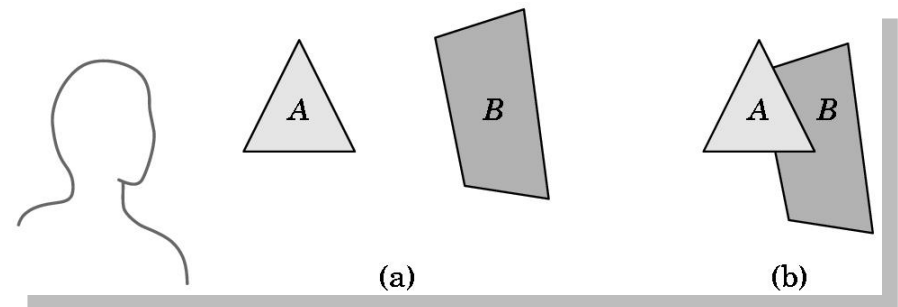
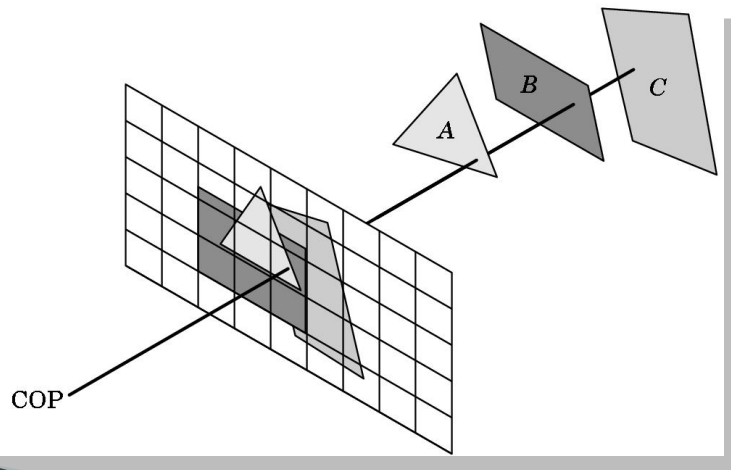
## ▶ Em ambientes complexos, poucas coisas são visíveis

- Quanto do mundo real é visto a cada instante?

# Espaço do objeto ou da imagem

## ► Abordagem

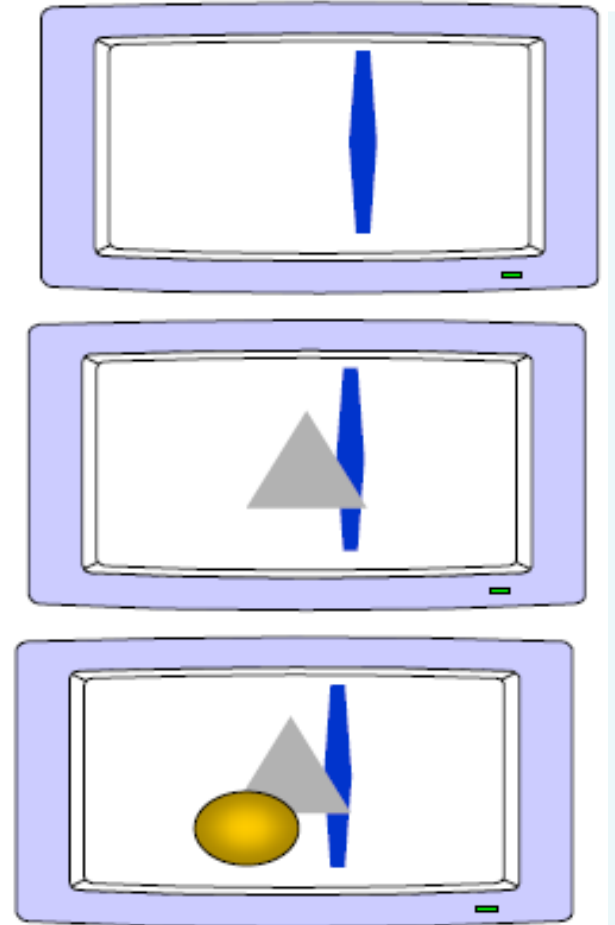
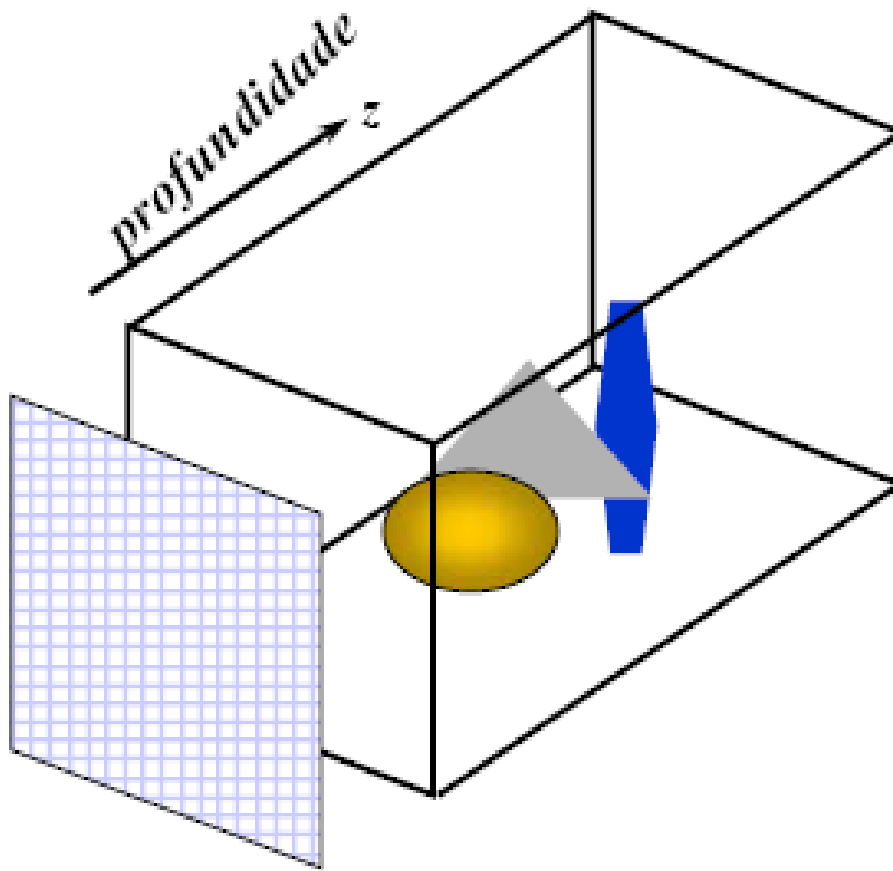
- Espaço da imagem: Ray tracing, Z-buffer, scan-line
- Espaço do objeto: painter's depth sorting, BSP



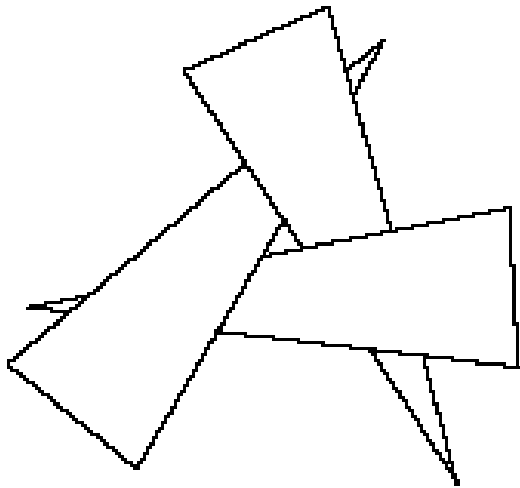
# Algoritmo do Pintor

- ▶ Algoritmo:
  - Escolha uma ordenação para os polígonos baseada em algum critério (e.g. profundidade de um ponto do polígono)
  - Renderize os polígonos nessa ordem, o mais distante primeiro
- ▶ Polígonos mais próximos são desenhados sobre os mais distantes

# Algoritmo do Pintor



# Algoritmo do Pintor



- ▶ Que ponto escolher para a ordenação?
- ▶ Dificuldade:
  - Se usado dessa forma, não funciona bem na maioria das geometrias
- ▶ Precisa de melhores maneiras de fazer a ordenação dos polígonos

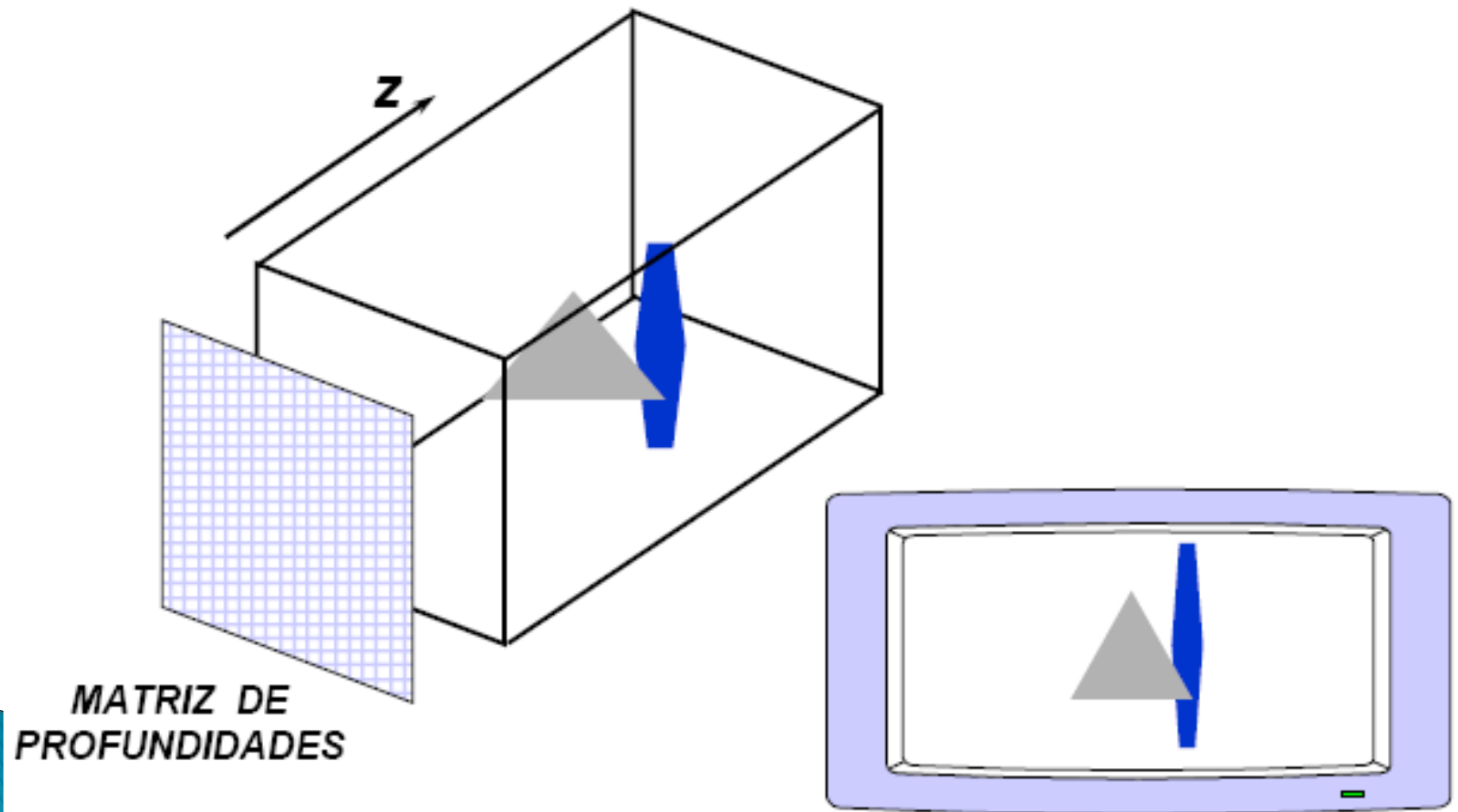
# Algoritmo Z-buffer

- ▶ Para cada pixel do display, tenha pelo menos 2 buffers:
  - Color buffer: Armazena a cor atual de cada pixel (o que será realmente mostrado)
  - Z-Buffer (ou depth buffer): armazena, para cada pixel, a profundidade do objeto mais próximo até então, a ser desenhado naquele pixel

# Algoritmo Z-buffer

- ▶ Inicialize o Z-buffer com um valor correspondente ao valor de Z mais distante visível ( $Z_{max}$ )
  - ▶ À medida que um polígono é preenchido, compute a profundidade de cada pixel a ser preenchido
    - If: profundidade < profundidade armazenada no z-buffer, preencha o pixel e armazene essa profundidade no z-buffer
    - Else: desconsidere o pixel (está atrás de algo que já foi desenhado).
- 

# Algoritmo Z-buffer



# Algoritmo Z-buffer

## ▶ Vantagens:

- Simples e hoje em dia implementado em hardware
- O Z-buffer é parte do que faz uma placa gráfica ser “3D”
- Computar as profundidades necessárias é simples

## ▶ Desvantagens:

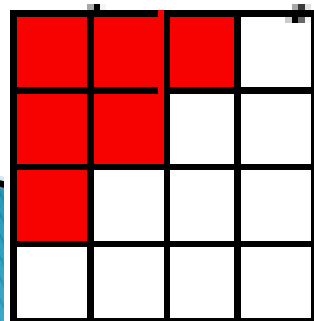
- Over-renders – pouco eficiente quando há grande número de polígonos
- Não trata transparência facilmente (precisaria guardar informação sobre polígonos parcialmente cobertos)

# Algoritmo A-buffer

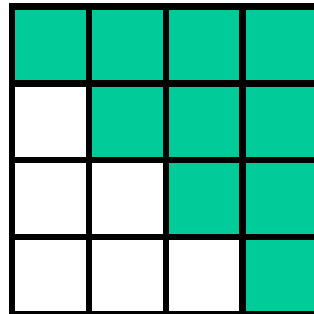
- ▶ Lida com superfícies transparentes e faz anti-aliasing
- ▶ Em cada pixel, mantenha um ponteiro para lista de polígonos ordenado por profundidade e uma máscara de cobertura de sub-pixels para cada polígono
  - Matriz de bits dizendo que partes do pixel está coberta

# Algoritmo A-buffer

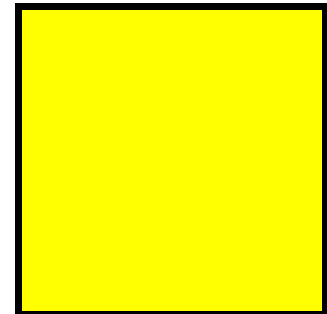
- ▶ Algoritmo: ao desenhar um pixel:
  - If: polígono é opaco e cobre o pixel, inserir na lista, removendo todos os polígonos mais distantes
  - If: polígono é transparente ou cobre apenas parte do pixel, inserir na lista, mas sem remover polígonos mais distantes
- ▶ Algoritmo: etapa de rendering
  - A cada pixel, percorra o buffer usando as cores dos polígonos e as máscaras de cobertura para fazer a composição:



sobre



=



# Algoritmo A-buffer

- ▶ Vantagem:

- Pode fazer mais que o Z-buffer
- Conceito de máscara de visibilidade pode ser usado em outros algoritmos

- ▶ Desvantagens:

- Não está em hardware, e é lento em software
- No fundo, é Z-buffer: mesmo problema de over-rendering

- ▶ Porém: usado em ferramentas de renderização de alta qualidade

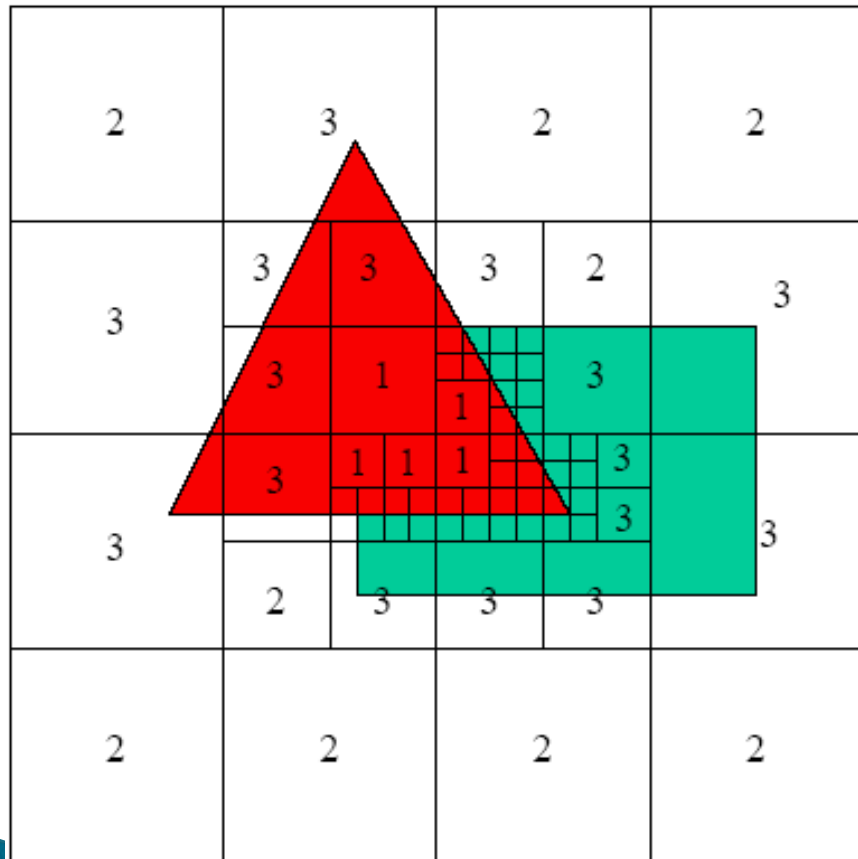
# Subdivisão de Áreas

- ▶ Explora a coerência de área: pequenas áreas de uma imagem tendem a ser cobertas por apenas um polígono
- ▶ Três casos triviais para determinar o que está na frente em uma região:
  - Um polígono está completamente à frente de qualquer coisa na região
  - Não há superfícies projetadas na região
  - Apenas uma superfície está completamente dentro da região, a corta ou envolve a região

# Algoritmo de Warnock

- ▶ Comece com imagem completa
- ▶ Se um dos 3 casos triviais ocorre, desenhe o que está na frente
- ▶ Caso contrário, subdivida a região recursivamente
- ▶ Se região chegou ao tamanho do pixel, escolha superfície com menor profundidade

# Algoritmo de Warnock



- ▶ Casos na imagem ao lado:
  1. Um polígono à frente
  2. Vazia
  3. Um polígono dentro, ao redor ou cortando a área

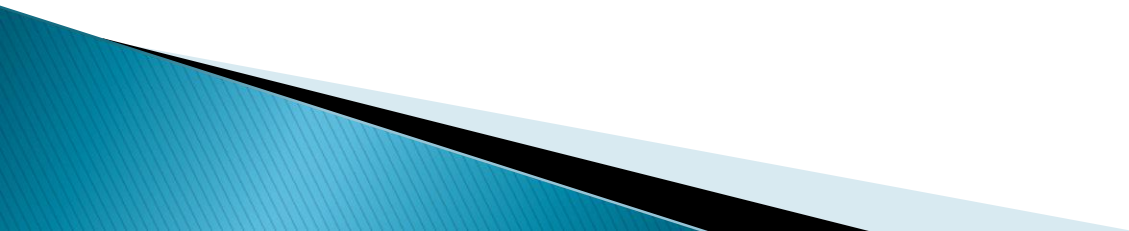
# Algoritmo de Warnock

## ▶ Vantagens:

- Não ocorre over-rendering
- Bom anti-aliasing: basta fazer mais uma subdivisão para obter informação de sub-pixel

## ▶ Desvantagem:

- Testes são complexos e lentos



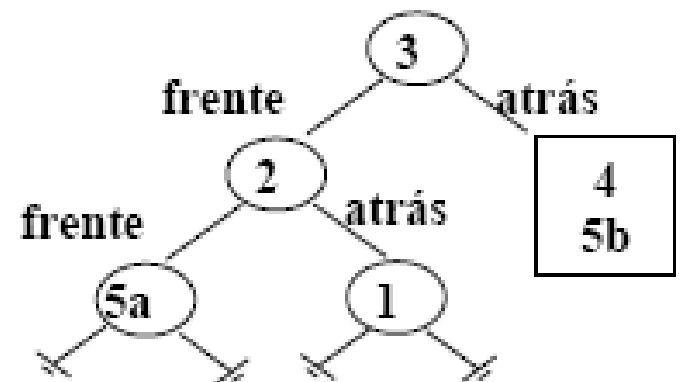
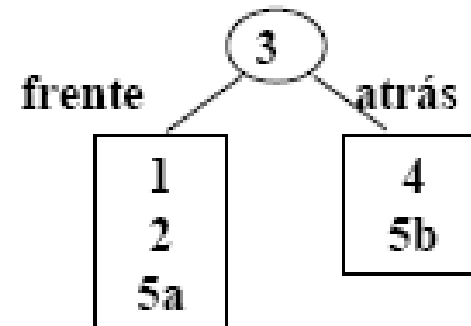
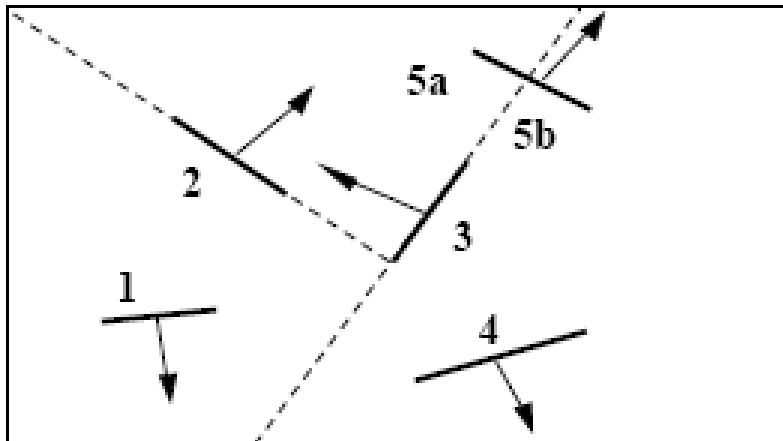
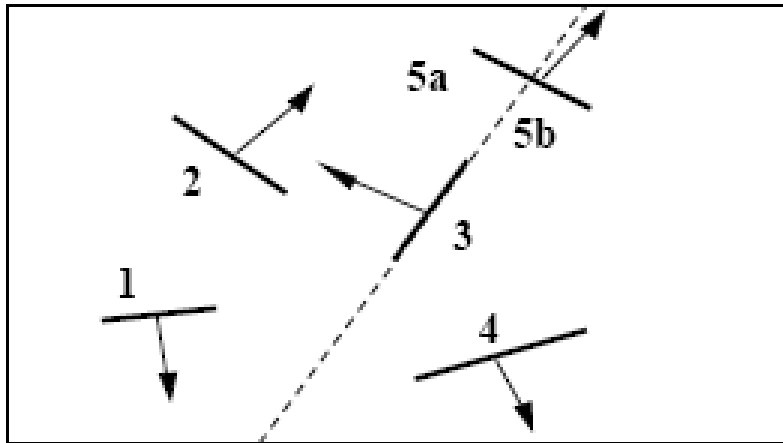
# BSP-Trees

## Binary Space Partion Trees

- ▶ Construção da BSP-Tree
  - Árvore fornece ordem de renderização
  - Árvore “quebra” o mundo 3D com planos
- ▶ Mundo é quebrado em células convexas
- ▶ Cada célula é a interseção de todos os “meio-espaços” dos planos de quebra na árvore
- ▶ Também usada em modelagem:
  - Nem sempre quando se fala em BSP se refere a este algoritmo

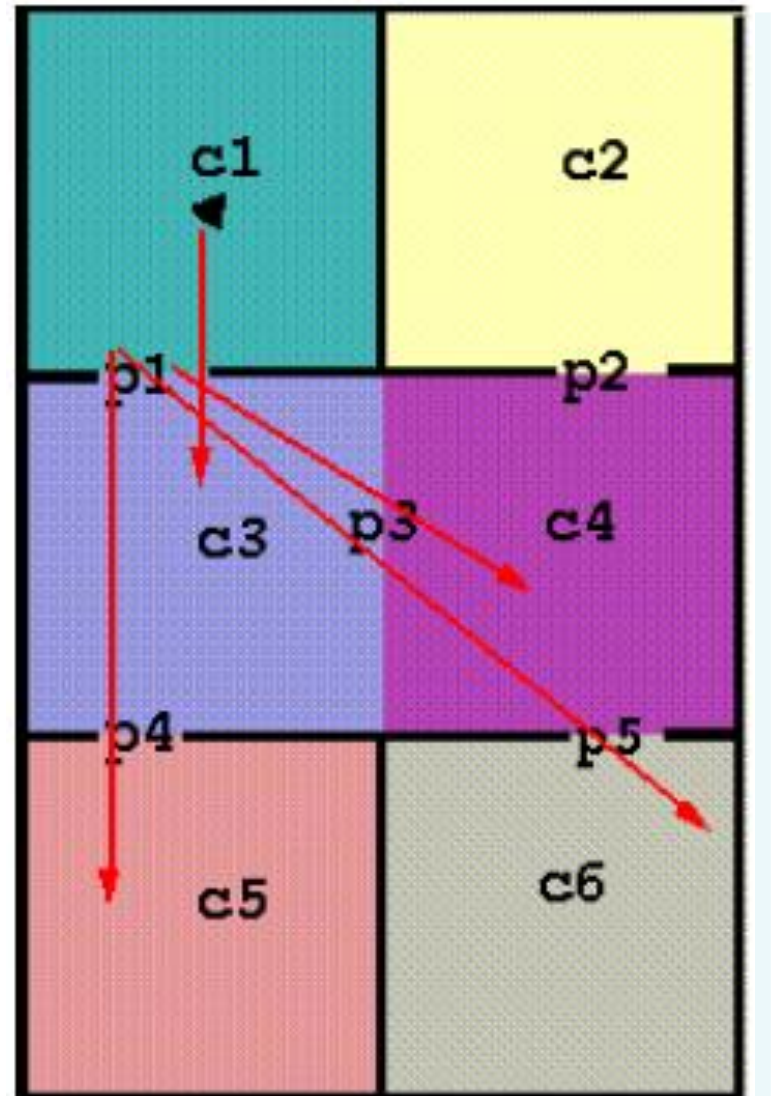
# BSP-Trees

## Binary Space Partion Trees

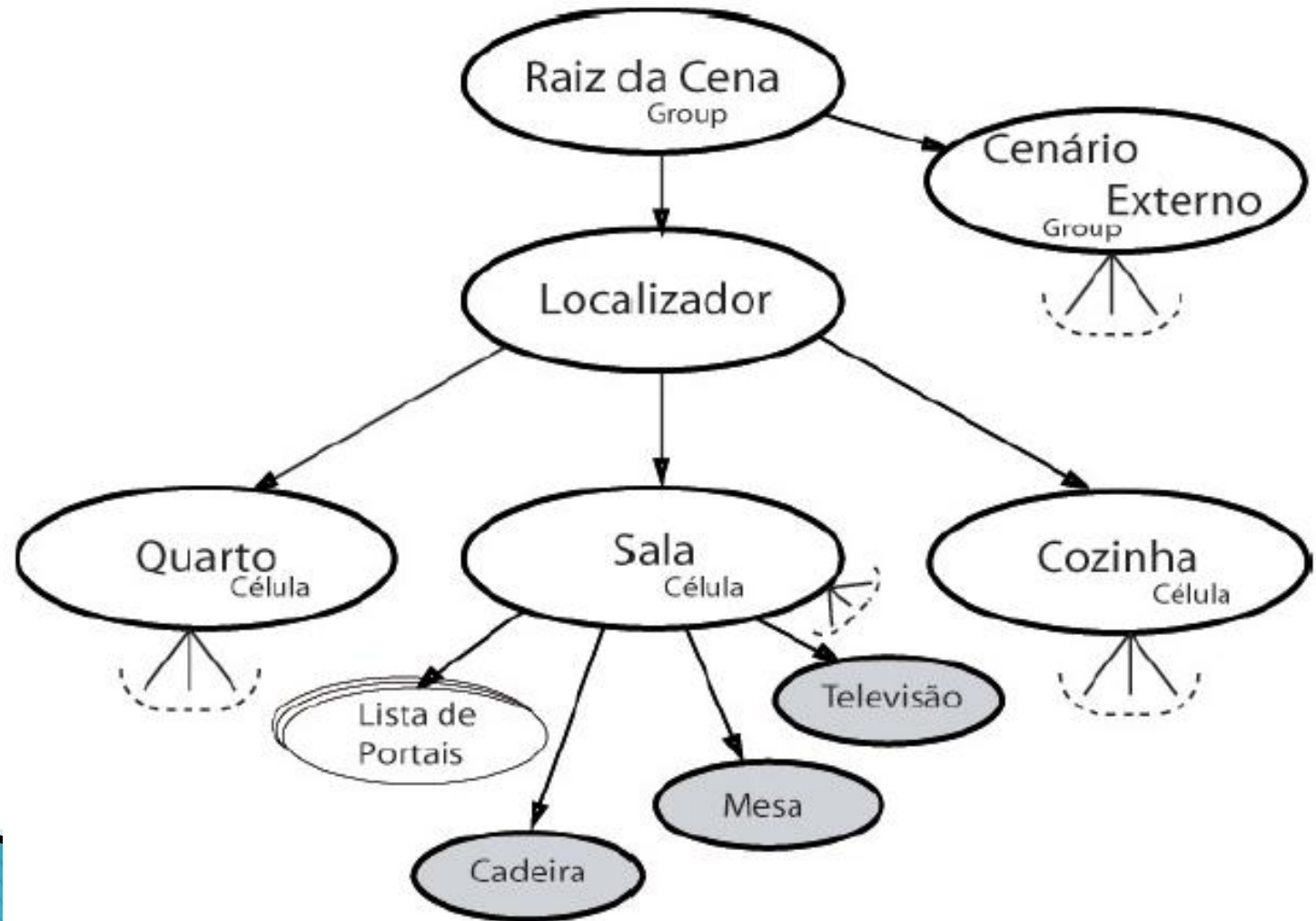


# Célula / Portal: Conceitos

- ▶ Em um ambiente fechado,
  - ▶ objetos presentes em quartos (células) distantes não podem ser vistos através das paredes (oclusores)
  - ▶ os objetos nos quartos adjacentes são vistos somente através das portas ou janelas (portais).



# Célula / Portal: Grafo de Cena



# Célula / Portal: Propriedades

## ▶ Vantagens

- Extremamente eficiente – apenas olha para células que estão realmente visíveis: visibility culling
- Facilmente modificável para visibilidade parcial: renderiza células parcialmente visíveis e deixa z-buffer fazer o resto
- Faz espelhos: crie “célula imaginária” espelhada e considere o espelho como portal

## ▶ Desvantagens

- Restrito a ambientes com boa estrutura para células/portais (ambientes fechados)